

Auditoria RADAR TASTYTRADE

AUDITORIA TÉCNICA INDEPENDENTE — RADAR TASTYTRADE PRO IA

Escopo: C:\Projetos Antigravity\RADAR-TASYTRADE **Data:** 08/09/2026 **Auditor:** Auditoria de Sistemas Sênior — revisão de qualidade, integridade, acuracidade e segurança de dados **Base de evidência:** 42 arquivos-fonte (~10.150 linhas TS/TSX), histórico Git (9 commits), configuração de build, ambiente e dependências **Commit auditado:** d11c895 + 10 arquivos modificados/não versionados em working tree

1. PARECER EXECUTIVO

VEREDITO: REPROVADO PARA USO EM DECISÃO FINANCEIRA REAL. Classificação de risco global: CRÍTICO (nota 2,8 / 10 em integridade de dados).

O sistema tem arquitetura de software competente — separação limpa domínio/serviço/apresentação, TypeScript strict, motores puros testáveis, zero segredo versionado. O problema não é engenharia. **O problema é que a camada de dados é majoritariamente sintética e o sistema não avisa o usuário disso.**

Em números:

Camada de análise	Origem real do dado	Status
Métricas de volatilidade (IVR, IVP, IV30)	API Tastytrade (real)	✓ ÚNICO dado real do sistema
Cotação spot / variação / 52w	Constante hardcoded no código	✗ Fabricado
Candles, MA20/50/200, RSI, MACD	Math.random() em runtime	✗ Fabricado e não-determinístico
GEX, Walls, Zero Gamma Flip	Curva exponencial sintética	✗ Fabricado
Prêmios das opções / crédito / max loss	% fixo do spot	✗ Fabricado
Smile de volatilidade / Skew	Fórmula paramétrica arbitrária	✗ Fabricado
Fundamentos US (66 ativos)	Constante hardcoded	✗ Fabricado
Fundamentos BR	API BRAPI (real) — com override manual para VALE3	⚠ Real, contaminado
Panorama macro (VIX, Fed, Score 58)	Texto fixo em JSX	✗ Fabricado
"Consultor IA"	if/else sobre String.includes()	✗ Não há IA

Aproximadamente 85% dos números exibidos ao usuário como análise quantitativa não têm origem em dado de mercado. A interface, o README e os textos didáticos afirmam o contrário — em vários pontos de forma explícita ("tempo real", "dados idênticos à plataforma", "não inventa número").

Isso é uma falha de **integridade e de veracidade**, não apenas de acurácia. Um sistema que erra o número tem bug. Um sistema que **apresenta número inventado como número medido** tem defeito de projeto — e, num produto de derivativos, exposição regulatória (CVM 20/2021, art. 16 — dever de fundamentação e transparência de fonte) e reputacional.

2. MATRIZ DE ACHADOS

#	Achado	Categoria	Severidade	Arquivo
A-01	Série de candles e indicadores técnicos gerados por Math.random()	Acuracidade	● CRÍTICO	us-market-data.ts:~100
A-02	GEX/Walls/Flip calculados sobre cadeia de opções sintética	Acuracidade	● CRÍTICO	tastytrade-market.service.ts:205-251
A-03	Cotação spot hardcoded em 3 datasets divergentes entre si	Integridade	● CRÍTICO	tastytrade-market.service.ts:157, us-market-data.ts, sp500-dataset.ts
A-04	Prêmios, crédito, max profit/loss e breakevens fabricados	Acuracidade	● CRÍTICO	volatility-engine.ts:190-280
A-05	Endpoints de API sem autenticação, rate limit ou validação	Segurança	● CRÍTICO	api/**/route.ts
A-06	Override hardcoded de fundamentos da VALE3	Integridade	● CRÍTICO	fundamentals-engine.ts:51-57, 82-88
A-07	Badge "TASTYTRADE LIVE API / tempo real" sobre dado estático	Integridade	● CRÍTICO	VolatilityAnalystView.tsx:212-215

A-08	Enriquecimento parcial: IV real + spot/walls obsoletos na mesma tese	Acuracidade	ALTO	VolatilityAnalystView.tsx:120-145
A-09	Heurística de unidade valor > 1 ? % : fração corrompe métricas	Acuracidade	ALTO	fundamentals-engine.ts:25-30
A-10	Breakeven do Iron Condor calculado por spot*0.94 / spot*1.05	Acuracidade	ALTO	volatility-engine.ts:~262
A-11	POP (probabilidade de lucro) hardcoded em 72% / 48%	Acuracidade	ALTO	volatility-engine.ts:~340
A-12	Max Loss do Iron Condor ignora a segunda asa	Acuracidade	ALTO	volatility-engine.ts:~250
A-13	Testes validam o valor fabricado, não a regra de negócio	Qualidade	ALTO	fundamentals-engine.test.ts:40-53
A-14	Fallback silencioso mascara falha de credencial/API	Integridade	ALTO	tastytrade-market.service.ts:128-149 , brapi.service.ts:150
A-15	Parser OCC incompatível com o símbolo gerado pelo próprio sistema	Qualidade	MÉDIO	symbol-parser.ts:24 vs tastytrade-market.service.ts:227
A-16	Leitor manual de .env.local em código de produção	Segurança	MÉDIO	tastytrade-auth.service.ts:38-51
A-17	ANTHROPIC_API_KEY no ambiente sem nenhum uso no código	Segurança	MÉDIO	.env.local
A-18	Token OAuth em arquivo plano na raiz do projeto	Segurança	MÉDIO	tasty_token.json
A-19	Cache de métricas global em módulo (vazamento entre requisições)	Qualidade	MÉDIO	tastytrade-market.service.ts:35
A-20	README declara "100% de cobertura" e "IA em tempo real"	Governança	MÉDIO	README.md
A-21	Sem CI, sem lint em pipeline, sem cobertura configurada	Qualidade	MÉDIO	ausência de .github/workflows
A-22	Divergência working tree x repositório (10 arquivos não commitados)	Governança	BAIXO	git status
A-23	.mp4 de 43 MB duplicado em raiz e public/	Qualidade	BAIXO	RADAR_PRO_QUANT.mp4

3. ACHADOS CRÍTICOS — DETALHAMENTO

A-01 — Análise técnica construída sobre ruído aleatório

src/lib/domain/us-market-data.ts, função generateCandlesticks:

```
const noise = (Math.random() - 0.48) * (currentSpot * 0.02);
const close = isLast ? currentSpot : Math.max(1, open + drift + noise);
...
const rsi = Math.min(85, Math.max(25, 50 + (close - ma20Accum) / (currentSpot * 0.05) * 20));
```

Os 90 candles diários, as médias 20/50/200, o RSI(14) e o histograma MACD são **gerados por passeio aleatório ancorado no spot fixo**. A partir daí o sistema deriva suporte, resistência, stop, alvo, R:R e um "checklist técnico CNPI-T de 5 itens" exibido como análise.

Três consequências, em ordem de gravidade:

- O usuário recebe um sinal técnico sem qualquer relação com o preço real do ativo.** Um RSI de 72 exibido na tela é ruído com escala, não sobrecompra.
- O resultado muda a cada carregamento da página.** Não há seed. Recarregar a tela produz outro gráfico, outro RSI, outro stop, outro alvo — para o mesmo ativo, no mesmo instante. Isso é irreproduzível e, portanto, **inauditável**: não existe forma de reconstituir a recomendação que o sistema deu ontem.
- A média móvel não é média móvel.** `ma20Accum = ma20Accum*0.95 + close*0.05` é uma EMA de $\alpha=0,05$ (≈ 39 períodos), rotulada como MA20. `ma50` é $\alpha=0,02$ (≈ 99 períodos) e `ma200` é $\alpha=0,005$ (≈ 399 períodos). Mesmo que a série fosse real, os três indicadores estariam errados por construção.

Risco assumido vs. mitigado: não há risco a assumir aqui. Isto se remove ou se substitui por OHLCV real — não existe versão aceitável de "indicador técnico sobre random walk" em produto financeiro.

A-02 / A-03 — Motor GEX alimentado por cadeia inventada, sobre spot inventado

tastytrade-market.service.ts:205-251 — o método chama-se `getGexAnalysis`, mas a variável interna chama-se `mockOptions`:

```
const baseGamma = Math.max(0.0005, (0.0055 - dist * 0.04));
const baseCall0i = Math.max(500, Math.round(35000 * Math.exp(-dist * 18) + (i >= 0 ? 15000 : 2000)));
```

Open Interest, gamma e IV de cada strike são uma exponencial decrescente em torno do spot. Ou seja: **as "Walls institucionais" são um artefato da fórmula, não do posicionamento de mercado**. Elas cairão sempre na mesma distância relativa do spot, para qualquer ativo, em qualquer dia.

O `gex-engine.ts` em si está **correto e bem escrito** — a fórmula Dollar GEX = $\Gamma \times \text{OI} \times S^2 \times 100$ está certa, a convenção de sinal (put negativa) está certa, a interpolação do Zero Gamma Flip está certa. É um bom motor recebendo lixo. Isso agrava o problema: a saída tem aparência institucional legítima.

Agravante de integridade (A-03): o spot vem de `getQuote()`, que é um dicionário fixo de 9 tickers (SPX: 6000.25, NVDA: 142.50, F: 14.62) com fallback genérico de **US\$ 100,00 para qualquer ticker desconhecido**. Existem três fontes de spot no projeto, mutuamente inconsistentes:

Ticker	getQuote() - via AI-Consultant	getQuote() - via F	getQuote() - via SPX
NVDA	142,50	142,50	142,50
Ticker fora da lista	100,00	150,00 (via ai-consultant)	95,00

Um mesmo ativo tem três preços diferentes dependendo da aba aberta. Isso é falha de integridade referencial em sua forma mais direta.

A-04 / A-10 / A-11 / A-12 — A matemática financeira da recomendação está errada

`volatility-engine.ts`, montagem do Iron Condor:

```
const shortPutPrem = Number((spot * 0.012).toFixed(2));
const longPutPrem = Number((spot * 0.005).toFixed(2));
const shortCallPrem = Number((spot * 0.011).toFixed(2));
const longCallPrem = Number((spot * 0.004).toFixed(2));
```

Os prêmios não dependem de IV, de DTE, de strike ou de moneyness. São 1,2% / 0,5% / 1,1% / 0,4% do spot, sempre. Consequência aritmética, simulando NVDA a US\$ 142,50 com `step = 5`:

- Crédito líquido: $1,71 - 0,71 + 1,57 - 0,57 = \text{US\$ } 2,00$
- Largura da asa: **US\$ 5,00**
- Razão crédito/largura: **0,40** → o sistema exibe **"CONFORME (Crédito remunera o risco)"**

Um Iron Condor com asas de 5 pontos e crédito de 2,00 tem POP implícito de aproximadamente 55–60%, não os **72% que o sistema exibe fixos** (A-11). E a regra Tastytrade de 1/3 está codificada como `>= 0.30` (`meetsCreditRule`), não 0,333 — o sistema aprova estruturas que a própria regra que ele cita reprovaria.

Pior, dois erros estruturais de payoff:

Max Loss (A-12): `maxLoss = (width - netCredit) * 100`. Para um Iron Condor de quatro pernas, `width` é a largura de **uma** asa. A perda máxima real é a largura da asa testada menos o crédito — o que a fórmula acerta por acaso, **mas apenas porque as duas asas têm largura idêntica (step)**. Se a lógica de strikes evoluir para asas assimétricas (Jade Lizard #22 e Broken-Wing já estão no catálogo e são elegíveis pela árvore de decisão), o número passa a estar errado sem que nenhum teste acuse.

Breakeven (A-10):

```
const lowerBreakeven = isCredit ? Number((spot * 0.94 - netCredit).toFixed(2)) : ...;
const upperBreakeven = isCredit ? Number((spot * 1.05 + netCredit).toFixed(2)) : ...;
```

O breakeven é calculado a partir de percentuais fixos do spot (−6% / +5%), **ignorando completamente os strikes das pernas que o próprio motor acabou de montar**. O correto é `shortPut - crédito` e `shortCall + crédito`. Como os strikes são ancorados nas Walls (que podem estar a 2%, 4% ou 8% do spot), o breakeven exibido pode divergir do real em vários pontos percentuais — e essa é exatamente a informação que o painel apresenta como "as 4 perguntas de dinheiro" para o investidor iniciante.

Este é o achado de maior potencial de dano financeiro direto: o usuário toma decisão de risco lendo um ponto de equilíbrio que não corresponde à estrutura descrita logo acima na mesma tela.

A-06 — Override hardcoded de emissor específico no motor fundamentalista

`fundamentals-engine.ts:51-57`:

```
if (symbol === 'VALE3') {
  effectiveRoe = 16.5;
  effectiveNetMargin = 28.5;
  effectivePE = 6.2;
```

```
}
```

E em `:82-88`, a dívida líquida/EBITDA é forçada a `0.8`. E em `brapi.service.ts:118-124`, há um terceiro override para o mesmo ticker.

A intenção é defensável — normalizar impairment não-caixa e excluir provisões Brumadinho/Samarco e IFRS-16 é prática correta de análise.

A implementação não é. Três problemas:

1. **É um número fixo, não um cálculo.** Quando a VALE divulgar o próximo trimestre, o sistema continuará exibindo ROE de 16,5% e P/L de 6,2 indefinidamente, com o rótulo `NORMALIZADO_FCO` sugerindo derivação metodológica.
2. **Ativa-se por condição frágil:** `symbol === 'VALE3' && effectiveRoe < 8`. Se a BRAPI retornar ROE de 7,9% legítimo num trimestre ruim de verdade, o sistema o substitui por 16,5% e **aprova** a empresa.
3. **A lógica está espalhada em três arquivos** com valores que precisam ser mantidos em sincronia manual — `brapi.service.ts`, `fundamentals-engine.ts` e `ai-consultant.ts:68-79` (que injeta ROE 0,0442 / P/L 32,29 / impairment 25,1 bi fixos).

Do ponto de vista de auditoria: **um motor de score que contém valores fixos por emissor não é um motor de score.** Se este comportamento se generalizar para outros tickers, o crivo CNPI-P perde qualquer validade estatística.

A-05 — Superfície de API totalmente aberta 🚫

Quatro rotas em `src/app/api/`, nenhuma com autenticação, autorização, rate limit, validação de schema ou verificação de origem. O projeto está publicado no Vercel (commit `976ffef`) e apontado para `github.com/jrveloso8-ai/tastyradar`.

`POST /api/avatar/generate` é o vetor mais grave. Aceita `text` arbitrário de qualquer origem e dispara geração de vídeo no D-ID, serviço **pago por crédito**, com espera síncrona de até 25 s por chamada. Sem rate limit e sem autenticação, isso é:

- **Exaustão financeira:** um script trivial esgota o saldo D-ID em minutos. Custo direto, não teórico.
- **Negação de serviço:** cada requisição segura um worker serverless por 25 s. Um punhado de requisições concorrentes derruba o app.
- **Uso indevido de identidade:** qualquer terceiro pode gerar vídeos de uma apresentadora com voz brasileira dizendo texto arbitrário, hospedado sob os créditos do titular da conta.
- **Injeção de parâmetro:** `sourceUrl` e `voiceId` do corpo da requisição vão direto ao payload do D-ID sem validação — o atacante escolhe a imagem-fonte.

`GET /api/market/metrics` expõe, sem autenticação, dados obtidos com o **token OAuth pessoal do titular** — transformando a conta Tastytrade dele em proxy público de market data. Além do custo de rate limit da API, isso pode configurar violação dos termos de uso da corretora.

`GET /api/market/fundamentals` propaga `symbol` do usuário direto para a URL da BRAPI e, em erro, devolve `error.message` cru ao cliente (vazamento de detalhe interno).

A-07 / A-08 / A-14 — O sistema declara "ao vivo" o que não é 🚫🚫

Três mecanismos que, combinados, produzem desinformação ativa:

(1) O badge. `VolatilityAnalystView.tsx:212-215` exibe **"TASTYTRADE LIVE API"** com tooltip *"Conectado diretamente à API Oficial da Tastytrade em tempo real (dados idênticos à plataforma)"*. O flag que o aciona é `json.live`, verdadeiro quando **qualquer** símbolo do lote retornou da API. Basta um ticker ao vivo para carimbar toda a grade como ao vivo.

(2) O enriquecimento parcial (A-08). O merge só sobrescreve `ivr`, `ivp`, `iv30`, `liquidityRating` e `daysToEarnings`. Permanecem estáticos: `spot`, `change`, `rv20`, `netGex`, `zeroGammaFlip`, `putWall`, `callWall`. O motor então calcula $VRP = IV30(\text{hoje}) - RV20(\text{fixo})$ e ancora os strikes em Walls congeladas em torno de um spot de setembro/2026.

Este é o **pior modo de falha possível**, pior do que dado 100% falso: o resultado é uma tese com metade de dado real, que passa em qualquer inspeção superficial, e cujo erro é invisível ao usuário. Dado sintético declarado é limitação. Dado sintético misturado com dado real e rotulado como real é armadilha.

(3) Os fallbacks silenciosos (A-14). Se o token OAuth expirar, `getMarketMetrics` captura a exceção com `console.warn` e devolve `source: 'preset-fallback'` — $IV30 = 25,0$, $\beta = 1,0$, liquidez = 4 para todos. O `brapi.service.ts:150` faz `catch { return { symbol } }`, transformando falha de rede em "empresa sem dados". O campo `source` existe no tipo e está corretamente preenchido — **e a interface não o exibe em lugar nenhum**. A informação de proveniência é coletada e depois descartada.

Some-se a isso o texto do próprio produto (`volatility-engine.ts`, seção `whatItDoesNotDo`): *"Não inventa número: é a regra mais rígida. Sem o dado real de cotação ou profundidade, entrega a condição matemática necessária."* O sistema afirma explicitamente ao usuário a garantia que mais viola.

4. ACHADOS DE SEVERIDADE ALTA E MÉDIA

A-09 — Heurística de unidade corrompe métricas legítimas 🚫

```
raw.returnOnEquity > 1 ? raw.returnOnEquity : raw.returnOnEquity * 100
```

O padrão se repete para ROE, margem líquida, margem EBITDA e dividend yield. A premissa — "se é maior que 1, já está em %" — quebra em duas direções:

- **Falso negativo:** ROE legítimo de **0,8%** (empresa em recuperação) chega como `0.8`, é interpretado como fração e vira **80%** → status BOM, 40 pontos. Uma empresa quase sem rentabilidade é pontuada como excelente.
- **Falso positivo:** DY de **1,5%** vindo como `1.5` é lido como já-percentual (correto por acaso); mas DY vindo como fração `0.015` vira 1,5% (também correto). Já um ROE de **150%** (AAPL: 145%, MA: 160% no próprio dataset) vindo como fração `1.50` seria lido como 1,5%.

A correção não é ajustar o limiar. É **fixar o contrato de unidade na fronteira do adaptador** (`brapi.service.ts`) e o motor sempre receber decimal.

A-13 — A suíte de testes valida a fabricação 🍷

`fundamentals-engine.test.ts:47-53:`

```
expect(debtMetric?.value).toBe(0.8);
expect(roeMetric?.value).toBe(16.5);
```

O teste afirma que o override hardcoded produz o valor hardcoded. É uma tautologia: **passa sempre e não pode falhar**, inclusive se toda a lógica de normalização for deletada e substituída por `return 16.5`. Cimenta o defeito A-06 no lugar em vez de detectá-lo.

Cobertura real da suíte (6 arquivos, ~40 asserções):

Módulo	Teste?	Observação
<code>gex-engine</code>	Parcial	Verifica sinal e existência; nenhuma asserção sobre o valor numérico do Dollar GEX
<code>volatility-engine</code>	Parcial	Testa roteamento de estratégia; zero teste de payoff, breakeven, max loss ou POP
<code>fundamentals-engine</code>	Tautológico	Ver acima
<code>symbol-parser</code>	OK	Único teste de qualidade decente do repositório
<code>sp500-dataset</code>	Trivial	Testa filtro <code><= 150</code> sobre constantes
<code>ai-consultant</code>	Ausente	355 linhas, zero teste
<code>did.service</code> / <code>brapi.service</code> / <code>auth.service</code>	Ausente	Zero teste, zero mock
Rotas de API	Ausente	Zero teste de contrato
Componentes React	Ausente	3.400 linhas de UI, zero teste

O README declara "**Vitest (100% de cobertura nos motores de cálculo)**". Não há `coverage` configurado no `vitest.config.ts` nem relatório no repositório. A afirmação é falsa e nunca foi medida.

A-15 — Incompatibilidade interna no símbolo OCC 🍷

O serviço emite `.${symbol}260918C${Math.round(strike * 1000)}` (`tastytrade-market.service.ts:227`). O parser (`symbol-parser.ts:24`) só divide por 1000 quando a string do strike tem **≥ 8 dígitos**. Para SPY a 595: $595 \times 1000 = 595000$ (6 dígitos) → o parser devolve **strike = 595.000,00**. Erro de 1000× num símbolo gerado pelo próprio sistema. Pior: `gex-engine.ts:133` emite o mesmo símbolo com `Math.round(s.strike)` — sem o $\times 1000$. Duas convenções incompatíveis para o mesmo formato, e o teste do parser só cobre a terceira.

A-16 / A-17 / A-18 — Higiene de segredos 🍷

- **A-16:** `tastytrade-auth.service.ts:38-51` implementa parser manual de `.env.local` como fallback, com `catch {}` mudo. É código server-side, então não vaza — mas é caminho de leitura de segredo fora do mecanismo do framework, invisível a qualquer varredura de segurança, e falha em silêncio.
- **A-17:** `ANTHROPIC_API_KEY` está no `.env.local` e **não é referenciada em nenhuma linha do código-fonte**. Segredo provisionado sem consumidor é dívida de segurança pura: expira sem ninguém notar, vaza sem ninguém detectar. Ou o "Consultor IA" deveria usá-la (e não usa — é `if/else`), ou ela deve ser revogada.
- **A-18:** `tasty_token.json` grava o access token OAuth em texto plano na raiz do projeto, com permissão do sistema de arquivos. Está corretamente no `.gitignore` (verificado: nenhum segredo em nenhum commit do histórico — **este ponto é um acerto**), mas em ambiente serverless o filesystem é efêmero e não compartilhado entre instâncias: **o cache de token não funciona em produção**, gerando refresh a cada cold start e risco de rate limit no endpoint OAuth da corretora.

A-19 — Estado global em módulo 🍷

`private metricsCache = new Map()` numa instância singleton exportada em nível de módulo. Em ambiente serverless, o cache é por instância (imprevisível); em servidor persistente, é **estado compartilhado entre todos os usuários sem chave de tenant** e cresce indefinidamente — não há evicção, apenas expiração passiva. Um `Map` que só recebe `set` é um vazamento de memória lento.

A-20 / A-21 / A-22 — Governança 🍷

- README afirma "100% de cobertura" (falso), "Consultor IA em Tempo Real" (é regex), "Smile de Volatilidade em tempo real" (é fórmula paramétrica), "GEX em tempo real via Tastytrade" (é `mockOptions`).
- Sem `.github/workflows`: nenhum CI, nenhum lint obrigatório, nenhum gate de teste antes de deploy. O `atualizar_github.bat` faz `commit-e-push` automático com mensagem gerada por timestamp — **o histórico Git não tem valor de auditoria** (3 dos 9 commits são "Update: Atualizacao do Radar ...").
- 10 arquivos divergentes entre working tree e repositório, incluindo módulos inteiros não versionados (`sp500-dataset.ts`, rota `/api/market/metrics`). O que está publicado no Vercel não é o que está no Git nem o que está na máquina.

5. O QUE ESTÁ CORRETO — E DEVE SER PRESERVADO

Auditoria honesta registra o que funciona. Este projeto tem fundação real:

1. **Arquitetura hexagonal bem executada.** `domain/` puro e sem I/O, `services/` isolando integrações, `components/` só apresentação. Isso é o que torna a correção viável — os defeitos estão concentrados na fronteira de dados, não espalhados.
2. **Motor GEX matematicamente correto.** $\text{Dollar Gamma} = \Gamma \times \text{OI} \times S^2 \times 100$ confere; sinal negativo para puts confere; interpolação linear do Zero Gamma Flip entre strikes de sinal oposto confere; Max GEX Magnet por magnitude absoluta confere. **Troque a entrada e a saída passa a valer.**
3. **Higiene de segredos no versionamento.** `.gitignore` abrangente e — verificado commit a commit em todo o histórico — **nenhuma credencial jamais versionada**. Isso é raro e é mérito.
4. **TypeScript em modo strict**, tipos de domínio explícitos, sem `any` disperso nos motores, zero uso de `dangerouslySetInnerHTML`, `eval` ou `innerHTML` — superfície de XSS nula.
5. **O campo source já existe** (`'tastytrade-live'` | `'preset-fallback'`, `'BRAPI_CONTABIL'` | `'NORMALIZADO_FCO'` | ...). A infraestrutura de rastreabilidade de proveniência **já está construída** — só não está sendo exibida. Essa é a correção de maior impacto e menor custo do plano abaixo.
6. **A camada didática é genuinamente boa.** As "4 perguntas de dinheiro", a analogia do seguro, a justificativa strike a strike, o playbook de 50% / 21 DTE — isso é conteúdo de qualidade profissional e é o diferencial competitivo real do produto. Ele só precisa estar apoiado em números verdadeiros.

6. RISCO: ONDE ASSUMIR, ONDE MITIGAR

Risco	Assumir ou mitigar	Racional
Dado sintético em ambiente educacional/demo	ASSUMIR — desde que rotulado	Simulador tem valor pedagógico legítimo. O defeito é o rótulo, não o dado.
Dado sintético apresentado como "ao vivo"	MITIGAR — bloqueante	É desinformação. Nenhuma justificativa de custo, prazo ou MVP sustenta.
Erro de payoff (breakeven, max loss, POP)	MITIGAR — bloqueante	Dano financeiro direto e imediato ao usuário.
Cobertura de dados incompleta (nem todo ticker com dado real)	ASSUMIR	Aceitável exibir "dado indisponível" e não gerar tese. Preferível a preencher com preset.
Latência maior por buscar dado real de opções	ASSUMIR	2-5 s de espera é preço trivial por número verdadeiro.
API sem autenticação	MITIGAR — bloqueante	Custo financeiro direto (D-ID), exposição de conta da corretora.
Override hardcoded VALE3	MITIGAR	Ou vira cálculo com fonte, ou é removido e o número bruto é exibido com ressalva.
Ausência de CI	MITIGAR — barato	GitHub Actions com <code>tsc --noEmit + vitest</code> custa 30 min de setup.
Indicadores técnicos aleatórios	REMOVER	Não há versão mitigada. Ou vem OHLCV real, ou a aba sai do ar.
Falta de teste em componentes React	ASSUMIR	Prioridade baixa frente ao restante. UI quebrada é visível; número errado não.

7. PLANO DE REMEDIAÇÃO

Fase 0 — Contenção imediata (24-48 h, bloqueante para qualquer uso)

1. **Tirar o deploy público do ar** ou colocá-lo atrás de senha (Vercel Password Protection, 5 min) enquanto A-05 não estiver resolvido. Enquanto estiver aberto, o custo de D-ID é passivo em aberto.
2. **Revogar** `ANTHROPIC_API_KEY` e `DID_API_KEY` e reemitir. Chave sem uso rastreado deve ser tratada como comprometida.
3. **Trocar o badge "TASTYTRADE LIVE API"** por um seletor de três estados por campo: `A0_VIVO` / `CACHE (Xs)` / `SIMULADO`. O campo `source` já carrega a informação — é exibi-lo.
4. **Banner global fixo enquanto durar a remediação:** *"Dados de cotação, gráfico e cadeia de opções são simulados. Não utilize para decisão de investimento."*

Fase 1 — Integridade dos dados (2-3 semanas)

5. **Cotação real:** substituir `getQuote()` pelo endpoint de quotes da Tastytrade (ou DXLink — a dependência `ws` já está no `package.json`)

e nunca foi usada). Eliminar os três datasets de spot concorrentes; fonte única.

6. **Cadeia de opções real:** trocar `mockOptions` pelo endpoint `/option-chains/{symbol}/nested` + greeks do streamer. O `gex-engine` não muda uma linha.
7. **OHLCV real:** deletar `generateCandlesticks` por completo. Sem série histórica real, a aba técnica não é exibida — não há meio-termo aceitável.
8. **Regra "sem dado, sem tese":** quando a fonte real falhar, retornar `null` com motivo e a UI exibir estado vazio explícito. **Eliminar todo fallback com preset numérico** (`tastytrade-market.service.ts:133-149`, `brapi.service.ts:150`, `getSP500Asset` fallback de US\$ 95, `ai-consultant.ts:40-61` fallback de US\$ 150).
9. **Fixar contrato de unidade** na fronteira do adaptador e remover as heurísticas `> 1 ? x : x*100` do motor (A-09).

Fase 2 — Correção matemática (1 semana, paralelizável com a Fase 1)

10. **Breakeven a partir dos strikes reais:** `shortPut - crédito / shortCall + crédito`. Nunca de percentual do spot.
11. **Max Loss por asa:** `max(larguraPut, larguraCall) - crédito`, preparado para estruturas assimétricas.
12. **POP calculado**, não constante: aproximação por delta da perna curta (`POP ≈ 1 - |Δshort|` para vertical de crédito; `1 - Δcall - |Δput|` para condor), com o delta vindo da cadeia real.
13. **Prêmios da cadeia real** (mid do bid/ask). Se não houver book, a estrutura **não é recomendada** — coerente com a promessa que o produto já faz ao usuário.
14. **Corrigir a regra do crédito** para `>= 1/3` (0,3333) e unificar as duas convenções de símbolo OCC (A-15).

Fase 3 — Governança (1 semana)

15. **Remover os overrides de VALE3** dos três arquivos. Substituir por normalização paramétrica com o impairment vindo da fonte (`nonRecurringImpairment` já existe no tipo), exibindo lado a lado valor contábil e valor normalizado — a estrutura `rawAccountingValue / adjustmentReason` já está pronta para isso.
16. **Reescrever os testes:** golden tests numéricos para o GEX (Dollar GEX de entrada conhecida), testes de payoff (crédito, breakeven, max loss verificados por cálculo independente), testes de contrato das rotas, e remoção dos testes tautológicos.
17. **CI no GitHub Actions:** `tsc --noEmit + vitest run --coverage + next build` obrigatórios antes do merge. Aposentar o `atualizar_github.bat`.
18. **Autenticação nas rotas:** middleware com sessão ou chave, rate limit por IP (`@upstash/ratelimit` no edge), validação de schema com Zod, e allowlist para `sourceUrl/voiceId`.
19. **Corrigir o README:** remover "100% de cobertura", "IA em tempo real", "GEX em tempo real" enquanto não forem verdade.

8. RECOMENDAÇÃO FINAL

Não coloque este sistema diante de terceiros — nem em demonstração comercial — antes de concluir a Fase 0 e a Fase 1. O risco não é o de um software com bugs; é o de um software que produz um número plausível, bem formatado e falso, sobre o qual alguém decide alocar capital em derivativos. A qualidade da apresentação agrava o risco em vez de atenuá-lo: quanto mais institucional parece a saída, menor a chance de o usuário questioná-la.

Dito isso, e sendo específico sobre o caminho: **a decisão certa é remediar, não reescrever.** A arquitetura está correta, o motor GEX está correto, o conteúdo didático é o ativo real do produto e a infraestrutura de rastreabilidade de fonte já está no código. O trabalho concentra-se em quatro pontos bem delimitados — substituir a fonte de dados, corrigir quatro fórmulas de payoff, remover os fallbacks silenciosos e fechar as rotas de API. Estimativa: **4 a 6 semanas** de trabalho focado. Uma reescrita custaria três vezes isso e jogaria fora o que está bom.

Ordem de execução inegociável: **Fase 0 hoje. Fase 1 antes de qualquer nova funcionalidade.** Nenhuma feature nova deve entrar antes de o sistema saber distinguir, para cada número que exibe, se ele foi medido ou inventado — e dizer isso ao usuário na própria tela.

Um último ponto, de princípio: o produto já promete ao usuário, por escrito, que *"não inventa número — é a regra mais rígida"*. Essa frase é a especificação correta do sistema. **Faça o código honrar o texto que já está nele.**

Lauda elaborado por análise estática integral do código-fonte, do histórico de versionamento, da configuração de build e do ambiente. A suíte de testes não pôde ser executada nesta sessão (node_modules compilado para Windows, incompatível com o ambiente de auditoria Linux) — as conclusões sobre cobertura derivam da leitura direta dos arquivos de teste, não de relatório de execução. Nenhuma credencial foi lida, transcrita ou transmitida; a verificação de vazamento em bundle foi feita por comparação local.